

1 Transformer Models

• Natural Language Processing (NLP)

– Tasks

- * **Classifying whole sentences:** Getting the sentiment of a review, detecting if an email is spam, determining if a sentence is grammatically correct or whether two sentences are logically related or not
- * **Classifying each word in a sentence:** Identifying the grammatical components of a sentence (noun, verb, adjective), or the named entities (person, location, organization)
- * **Generating text content:** Completing a prompt with auto-generated text, filling in the blanks in a text with masked words
- * **Extracting an answer from a text:** Given a question and a context, extracting the answer to the question based on the information provided in the context
- * **Generating a new sentence from an input text:** Translating a text into another language, summarizing a text

– Rise of LLMs

- * Perform a wide variety of the above language tasks without task-specific training
- * Scale, general capabilities, in-context learning, emergent abilities
- * Limitations:
 - Hallucinations
 - Lack of true understanding
 - Reflecting training data bias in outputs
 - High computational and environmental costs

• Transformers are everywhere!

– NLP, Computer Vision, Audio Processing, Multimodal Models

```

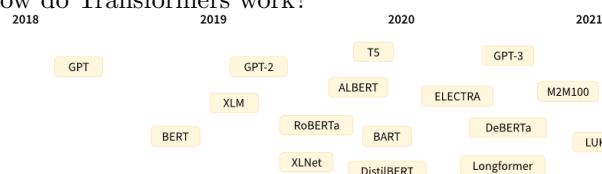
1 from transformers import pipeline
2
3 classifier = pipeline("sentiment-analysis")
4 classifier(
5     ["I've been waiting for a HuggingFace course my whole
6     life.", "I hate this so much!"])
7
8 [{ 'label': 'POSITIVE', 'score': 0.9598047137260437 },
9  { 'label': 'NEGATIVE', 'score': 0.9994558095932007 }]

```

– text-generation, text-classification, summarization, translation, zero-shot-classification, feature-extraction, image-to-text, image-classification, object-detection, automatic-speech-recognition, audio-classification, text-to-speech, image-text-to-text

– Model Hub: <https://huggingface.co/models>

• How do Transformers work?



– Carbon Footprint

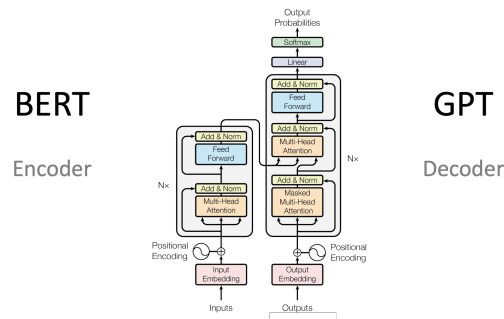
- * $CO_2 = f(\text{type of energy, training time, hardware, region})$
- * Use pretrained models, fine-tuning, smaller experiments + debugging, literature review of hyperparameter ranges, random search vs. grid search
- Transfer Learning is the act of initializing a model

with pretrained weights, then fine-tuning it on a downstream task.

- * Pretraining objective: Predict the next word in a sentence
- * Pretraining objective: Guess the value of randomly masked words
- * Fine-tuning: Further training the model on a smaller, task-specific dataset
- * Always try to leverage a pretrained model first!

• How Transformers solve tasks

- Language models are trained to predict the probability of a word given context of surrounding words
 1. Masked language modeling (MLM): Randomly mask some tokens and train model to predict the original tokens based on surrounding context. This allows model to learn bidirectional context (i.e. BERT).
 2. Causal language modeling (CLM): Train model to predict the next token in a sequence given all previous tokens (i.e. GPT).



– Types of language models

1. Encoder-only models (i.e. BERT): Understand context from both directions. For tasks requiring deep understanding of text, such as classification, NER, and Q&A.
2. Decoder-only models (i.e. GPT): Process text from left-to-right. Great at text generation tasks such as sentence completion, creative writing, code generation.
3. Encoder-Decoder models (i.e. T5, BART): Combine both approaches. Great at sequence-to-sequence tasks like translation, summarization, and Q&A.

– Tasks

- * Text Generation: output from decoder passed to language modeling head, which does linear transformation to convert hidden states into logits. Cross-entropy loss calculated between shifted logits and labels to output most likely token
- * Text Classification: Pretrained BERT + sequence classification head (transforms hidden states into logits). Cross-entropy between logits and labels to find most likely label
- * Token Classification: Pretrained BERT + token classification head (transforms hidden states into logits). Cross-entropy loss between logits and each token to find most likely label
- * Question Answering: Pretrained BERT + Span classification head (transforms hidden states into start and end logits). Cross entropy loss between

logits and label position to find most likely span of text.

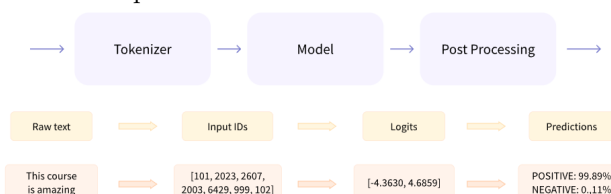
- * Summarization: Pretrained BART + seq2seq language modeling head (linear transformation to convert decoder hidden states into logits). Cross-entropy loss between shifted logits and labels to output most likely token
- * BART encodes source language into an embedding, then decoder generates target language
- * Speech and audio: Whisper model transcribes audio into text and does translation using encoder-decoder architecture. Whisper is trained on 680,000 hours of labeled audio data
- * Computer Vision: Two approaches
 - Vision Transformer (ViT): Splits image into patches, flattens them, and processes them as a sequence using transformer architecture
 - Hybrid models (i.e. ConvNeXT): Use CNN layers to extract features from images, then feed these features into transformer layers for further processing

Transformer Architectures

Task	Suggested Architecture	Examples
Text classification (sentiment, topic)	Encoder	BERT, RoBERTa
Text generation (creative writing)	Decoder	GPT, LLaMA
Translation	Encoder-Decoder	T5, BART
Summarization	Encoder-Decoder	BART, T5
Named entity recognition	Encoder	BERT, RoBERTa
Question answering (extractive)	Encoder	BERT, RoBERTa
Question answering (generative)	Encoder-Decoder or Decoder	T5, GPT
Conversational AI	Decoder	GPT, LLaMA

2 Using Transformers

- How does Pipeline work?



```

1 import torch
2 from transformers import AutoTokenizer,
  AutoModelForSequenceClassification
3
4 checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
5 tokenizer = AutoTokenizer.from_pretrained(checkpoint)
6 model = AutoModelForSequenceClassification.from_pretrained(
7     checkpoint)
8 sequences = ["I've been waiting for a HuggingFace course my whole life.", "So have I!"]
9
10 tokens = tokenizer(sequences, padding=True, truncation=True,
11     return_tensors="pt")
12 output = model(**tokens)
  
```

- Tokenizers
 - Word-based - splits by spaces but often generates too many [UNK] tokens
 - Character-based - splits into characters but generates too many tokens
 - Subword-based - balances vocabulary size and coverage. For example, "tokenization" would be split into "token", "ization"

3 Fine-Tuning a Pretrained Model

- Fine-Tune on one batch

```

1 import torch
2 from torch.optim import AdamW
3 from transformers import AutoTokenizer,
  AutoModelForSequenceClassification
4
5 # Same as before
6 checkpoint = "bert-base-uncased"
7 tokenizer = AutoTokenizer.from_pretrained(checkpoint)
8 model = AutoModelForSequenceClassification.from_pretrained(
9     checkpoint)
10 sequences = [
11     "I've been waiting for a HuggingFace course my whole life.",
12     "This course is amazing!",
13 ]
14 batch = tokenizer(sequences, padding=True, truncation=True,
15     return_tensors="pt")
16
17 # This is new
18 batch["labels"] = torch.tensor([1, 1])
19
20 optimizer = AdamW(model.parameters())
21 loss = model(**batch).loss
22 loss.backward()
23 optimizer.step()
  
```

- Fine-Tune via the Trainer API

```

1 from datasets import load_dataset
2 from transformers import AutoTokenizer,
  DataCollatorWithPadding, TrainingArguments, Trainer
3
4 raw_datasets = load_dataset("glue", "mrpc")
5 checkpoint = "bert-base-uncased"
6 tokenizer = AutoTokenizer.from_pretrained(checkpoint)
7
8 # Tokenize samples in parallel
9 def tokenize_function(example):
10     return tokenizer(example["sentence1"], example["sentence2"],
11         truncation=True)
12 tokenized_datasets = raw_datasets.map(tokenize_function,
13     batched=True)
14
15 # Pad each sample to max length in batch
16 data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
17
18 def compute_metrics(eval_preds):
19     metric = evaluate.load("glue", "mrpc")
20     logits, labels = eval_preds
21     predictions = np.argmax(logits, axis=-1)
22     return metric.compute(predictions=predictions, references=
23         labels)
24
25 training_args = TrainingArguments("/path/to/saved/trained/
26     model", eval_strategy="epoch")
27 model = AutoModelForSequenceClassification.from_pretrained(
28     checkpoint, num_labels=2)
29
30 trainer = Trainer(
31     model,
32     training_args,
33     train_dataset=tokenized_datasets["train"],
34     eval_dataset=tokenized_datasets["validation"],
35     data_collator=data_collator,
36     processing_class=tokenizer,
37     compute_metrics=compute_metrics,
38 )
  
```

4 Classical NLP Tasks

- pass